

Plano de Correção Aplicado — Fluxyra (plano de 26/07)

Documento de fechamento das correções P0→P2 do plano de 26/07, implementadas no repositório `saas` (Next.js 16 / TypeScript, branch `feat/f3-piapi-landing-pricing`, deploy `https://5be3213ef.abacusai.cloud`, systemd `fluxyra.service` na porta 3000).

- **Build:** `npx tsc --noEmit` → OK · `npm run build` → OK
- **Serviço:** `systemctl is-active fluxyra.service` → `active`
- **Smoke test:** `GET /` → 200 · `GET /api/models` → 200 · `POST /api/webhooks/piapi` sem assinatura → 401
- **Nenhuma task paga foi criada na PiAPI durante a implementação.**

Item	Escopo	Status
§3.1	Webhook PiAPI + unificação do path de storage	IMPLEMENTADO
§3.3	Débito/estorno de créditos atômico e idempotente	PARCIAL (garantias ativas em código; RPC SQL criada mas não aplicada)
§3.2	Validações locais antes do débito	IMPLEMENTADO
§3.4	Timeout de gerações presas (30 min)	IMPLEMENTADO
§3.5	Higiene: dead code + documentação de divergências	IMPLEMENTADO

§3.1 — Webhook PiAPI + unificação do path de storage — IMPLEMENTADO

O que foi feito

1. **Registro do webhook no payload da PiAPI** — `src/lib/piapi/client.ts` (`buildVideoPayloadInner`, ~l.421-431).

Quando **ambas** as variáveis `PUBLIC_BASE_URL` e `PIAPI_WEBHOOK_SECRET` estão definidas, o payload passa a incluir:

```
ts
const config: Record<string, unknown> = { service_mode: "public" };
if (process.env.PUBLIC_BASE_URL && process.env.PIAPI_WEBHOOK_SECRET) {
  config.webhook_config = {
    endpoint: `${process.env.PUBLIC_BASE_URL}/api/webhooks/piapi`,
```

```

    secret: process.env.PIAPI_WEBHOOK_SECRET,
  };
}

```

Se as vars não existirem, o comportamento anterior é mantido (sem webhook) — o fluxo continua funcionando via polling da rota de status.

2. Variáveis de ambiente — `.env.local` (arquivo **gitignored**, não versionado):

- `PUBLIC_BASE_URL=https://5be3213ef.abacusai.cloud`
- `PIAPI_WEBHOOK_SECRET=<uuid v4 gerado, 36 chars>` (mascarado; não exposto neste relatório)

3. Rota de webhook reescrita — `src/app/api/webhooks/piapi/route.ts` :

- Lê `rawBody` via `req.text()` antes de qualquer parse (necessário para validar HMAC).
- `verifySignature(rawBody, headers)` aceita **duas** formas:
 - segredo em texto no header (`x-webhook-secret` / `x-piapi-secret` / `x-api-key`);
 - HMAC SHA256 hex em `x-signature` / `x-webhook-signature` / `x-piapi-signature`.
 - Comparação com `timingSafeEqual`. Se `PIAPI_WEBHOOK_SECRET` não estiver configurado, a verificação passa (fail-open apenas quando o operador não configurou segredo).
 - Parse tolerante (`parsed.data` aninhado **ou** payload plano); extrai `task_id`, `status`, `output`, `error`.
 - Busca a `generation` pelo `provider_task_id`.
 - **Idempotência**: se a `generation` já está `completed` / `failed`, retorna 200 sem reprocessar.
 - **status failed**: atualiza a `generation` e chama `refundCredits` (idempotente).
 - **status completed**: extrai a URL do vídeo, baixa o arquivo, faz upload para o bucket `assets` no path unificado e atualiza `result_url` + `status=completed`.
 - Estados intermediários apenas sincronizam status.
 - `auditLog` em todos os ramos.

4. Unificação do path de storage — webhook e rota de status agora gravam no mesmo padrão singular:

```

{user_id}/{type}/{generation_id}.{ext} → bucket "assets"
(src/app/api/webhooks/piapi/route.ts, l.176).

```

Decisão documentada: o plano sugeria o plural (`videos/`), mas a instrução pedia para verificar antes de decidir. A inspeção do bucket `assets` em produção mostrou que os dados **já existentes** usam pastas **singulares** (`video`, `image`, `audio`), gravadas pela rota de status ativa. Para manter consistência com os dados reais e com o writer já em produção, unifiquei no **singular** (ajustei o webhook de plural→singular). Sem impacto funcional: `result_url` guarda a URL pública absoluta.

Antes vs depois

- **Antes:** nenhum `webhook_config` no payload → a PiAPI nunca notificava; a conclusão dependia 100% do polling. O path de storage divergia entre webhook (plural) e status route (singular).
- **Depois:** a PiAPI notifica em `/api/webhooks/piapi` (quando as vars existem), com assinatura validada; conclusão/falha processadas via webhook **ou** polling (ambos idempotentes); path unificado.

Como testar (sem task paga)

- `POST /api/webhooks/piapi` sem assinatura → **401** (`{"error": "Assinatura inválida"}`).
- `POST /api/webhooks/piapi` com header `x-webhook-secret: <PIAPI_WEBHOOK_SECRET>` e corpo `{"task_id": "<inexistente>", "status": "completed"}` → 200/ok (generation não encontrada é tratada sem erro).
- Verificar no bucket `assets` que novas mídias caem em `{user_id}/video/{id}.mp4` .

§3.3 — Débito/estorno atômico e idempotente — PARCIAL

O que foi feito

1. **Helpers em código** — `src/lib/credits.ts` (novo):
 - `debitCredits(service, userId, jobId, amount, requestId)` :
débito via **compare-and-swap** (lê `credits_balance` , faz `update ... .eq("credits_balance", current)` com **retry**), garantindo que dois débitos concorrentes não corrompam o saldo. Registra o lançamento em `credit_transactions` com `reason: "generation"` e `related_job_id: jobId` . Retorna `{ ok, balance }` ou `{ ok:false, reason:"insufficient" }` .
 - `refundCredits(service, userId, jobId, amount?, requestId)` :
idempotente — antes de creditar, verifica se já existe lançamento `reason="refund"` para o mesmo `related_job_id` ; se existir, não credita de novo. Se `amount` não for passado, deriva do débito original (`reason="generation"`). Credita e registra lançamento `reason="refund"` .
 - Ambos usam o **service client** (contorna RLS de forma robusta).
2. **RPC SQL** — `supabase/migrations/0002_rpc_credits.sql` (novo):
funções `debit_credits` / `refund_credits` em PL/pgSQL, alinhadas ao schema de produção (reason `generation` / `refund`), para débito/estorno atômicos no banco.
3. **Substituição no fluxo** — `src/app/api/generate/video/route.ts` :
 - débito manual (select + update + insert de ledger) → `debitCredits` (l.222);
 - se `!ok` : marca a generation como `failed` e responde **402** (insufficient) ou **500** (outro erro), sem cobrar;
 - `catch` usa `refundCredits` idempotente (l.330), mantendo a classificação de erro 402/503/502.
 - `src/app/api/generate/status/route.ts` e o webhook também usam `refundCredits` .

Por que PARCIAL

A **RPC SQL não foi aplicada** ao banco hospedado: o Supabase deste projeto **não expõe DDL via service role** (a RPC `exec_sql` responde 404 e não há connection string Postgres disponível para rodar a migration). Portanto:

- A migration `0002_rpc_credits.sql` fica como **artefato a aplicar manualmente** (Supabase → SQL Editor → colar e executar).
- **Enquanto a RPC não é aplicada**, as garantias de atomicidade/idempotência rodam em `src/lib/credits.ts` (CAS com retry + check-then-act idempotente). O código **não** chama `supabase.rpc('debit_credits'...)` — isso quebraria em produção, pois a função ainda não existe no banco.

Antes vs depois

- **Antes:** débito = select saldo → update → insert ledger, sem proteção contra corrida e sem idempotência no estorno (risco de débito duplo/estorno duplo).
- **Depois:** débito com CAS+retry; estorno idempotente por `related_job_id`.

Como testar (sem task paga)

- Inspecionar `credit_transactions` após uma tentativa: deve haver **um** lançamento `reason="generation"` por generation, e no máximo **um** `reason="refund"` por job.
- Forçar duas chamadas de estorno para o mesmo `job_id` → apenas **um** crédito é devolvido (o segundo é no-op).
- Para ativar a versão em banco: aplicar `supabase/migrations/0002_rpc_credits.sql` no SQL Editor.

Limitações

- O estorno idempotente é **check-then-act** em código: sob concorrência extrema existe uma pequena janela teórica de corrida. A aplicação da RPC SQL (0002) elimina essa janela ao mover a lógica para dentro de uma transação no banco.

§3.2 — Validações locais antes do débito — IMPLEMENTADO

O que foi feito

- `src/lib/generation-validation.ts` (novo) — `validateVideoRequest(input, modelParams, requestId)` :
- **URLs de referência:** devem ser `http(s)` ; caso contrário → resultado inválido (a rota responde **400**). HEAD probe de 3s **não-bloqueante** (`Promise.allSettled`) — falha de rede na sondagem não reprova o request.
- **aspect_ratio:** se enviado, precisa constar na lista suportada pelo modelo (`params.supported_aspect_ratios` quando presente, senão defaults por backend: `seedance/kling/hailuo/veo3/wan/ltx`).
- **duration:** validada contra `dur_min` / `dur_max` dos params do modelo.
- Integração em `src/app/api/generate/video/route.ts` (l.154-171): a validação roda **antes** de qualquer débito; se inválida → **400** e **nenhum crédito é cobrado**.

Antes vs depois

- **Antes:** parâmetros inválidos só eram rejeitados pela PiAPI, **depois** do débito → usuário perdia crédito (dependia do estorno) e gastava uma chamada.
- **Depois:** rejeição local com **400** antes do débito.

Como testar (sem task paga)

- `aspect_ratio` inexistente para o modelo → **400** (sem débito).
- URL de referência não-http (ex.: `ftp://...` ou `data:`) → **400**.
- `duration` fora de `[dur_min, dur_max]` → **400**.

§3.4 — Timeout de gerações presas (30 min) — IMPLEMENTADO

O que foi feito

- `src/app/api/generate/status/route.ts` :
- `timeoutMin = Number(process.env.GENERATION_TIMEOUT_MINUTES) || 30` (l.77).
- `isStale` = generation com `created_at` mais antigo que `timeoutMin` (l.79).
- `timeoutIfStale()` (l.80): quando `isStale`, marca a generation como `failed`, chama `refundCredits` (idempotente) e retorna 200 com status `failed`.
- Chamado nos ramos “ainda processando” (l.290) e no `catch` do poll (l.305), antes de responder `processing` — assim uma generation travada é encerrada e estornada no próximo poll do cliente.
- O ramo `failed` também usa `refundCredits` (l.277).

Antes vs depois

- **Antes:** generation presa em `processing` ficava assim indefinidamente; crédito nunca era devolvido.
- **Depois:** após `GENERATION_TIMEOUT_MINUTES` (default 30), é marcada `failed` e estornada automaticamente.

Como testar (sem task paga)

- Em ambiente de teste, dar `UPDATE generations SET created_at = now() - interval '40 minutes'` em uma generation `processing` e chamar `GET /api/generate/status?id=...` → retorna `failed` e registra estorno em `credit_transactions`.
- Ou setar `GENERATION_TIMEOUT_MINUTES=0` temporariamente para forçar o timeout.

Limitações

- O timeout é acionado pelo **polling** da rota de status (não é um job cron). Uma generation só é encerrada quando o cliente (ou o webhook) tocar a rota novamente. Para expiração garantida sem polling seria necessário um cron/worker externo.

§3.5 — Higiene: dead code + documentação de divergências — IMPLEMENTADO

Dead code removido (`git rm`, 0 imports confirmados)

- `src/hooks/use-generation.ts`
- `src/components/studio/media-gallery.tsx`

Divergências de schema documentadas

- `supabase/migrations/0001_init.sql` **NÃO corresponde à produção**. É um schema “v2/fantasia” (nomes e colunas diferentes). A **fonte de verdade** é `supabase/SCHEMA_PRODUCAO.md`. A migration 0001 **não deve ser executada** contra o banco de produção. As novas funcionalidades (créditos, ledger, storage) foram todas alinhadas ao schema real:
- `profiles(id, email, credits_balance, stripe_customer_id, plan, plan_credits_monthly, created_at)`

- `credit_transactions(id, user_id, amount, reason[welcome/subscription/topup/generation/refund], related_job_id, created_at)`
- `generations(id, user_id, model_id, type, prompt, ..., status, result_url, provider_task_id, credits_used, error_message, created_at, updated_at)`
- `buckets`: `uploads` (referências enviadas) e `assets` (mídias geradas).

Duplicação de endpoints documentada (NÃO alterada)

- `/api/me` e `/api/credits` retornam essencialmente o mesmo dado (`{ credits: profile.credits_balance, plan }`). Recomenda-se unificar no futuro, mas **não foi alterado agora** para evitar regressão em consumidores existentes.

Arquivos alterados nesta entrega

Criados

- `src/lib/credits.ts`
- `src/lib/generation-validation.ts`
- `supabase/migrations/0002_rpc_credits.sql`
- `PLANO_CORRECAO_APLICADO.md` (este relatório)

Modificados

- `src/lib/piapi/client.ts` (`webhook_config`)
- `src/app/api/generate/video/route.ts` (validação + débito/estorno via helpers)
- `src/app/api/generate/status/route.ts` (timeout + estorno)
- `src/app/api/webhooks/piapi/route.ts` (reescrito)
- `.env.local` (`gitignored` — vars `PUBLIC_BASE_URL`, `PIAPI_WEBHOOK_SECRET`)

Removidos

- `src/hooks/use-generation.ts`
- `src/components/studio/media-gallery.tsx`

Commits (um por bloco)

1. feat: webhook piapi + unified storage path (§3.1)
2. feat: atomic credits debit/refund RPC (§3.3)
3. feat: local validation before debit (§3.2)
4. fix: generation timeout 30min (§3.4)
5. chore: remove dead code, document schema divergences (§3.5)

Observação sobre a granularidade dos commits: os arquivos

`video/route.ts` e `status/route.ts` concentram mudanças de mais de um bloco (validação + débito + timeout). Como o git versiona o arquivo inteiro, cada um foi incluído no commit do bloco predominante (video→§3.3, status→§3.4), com a menção às demais mudanças no corpo do commit.

Ações manuais pendentes (para o operador)

1. Aplicar `supabase/migrations/0002_rpc_credits.sql` no Supabase SQL Editor para ativar as RPCs atômicas no banco (o código já funciona sem isso, via CAS).

2. Confirmar que `PUBLIC_BASE_URL` e `PIAPI_WEBHOOK_SECRET` estão presentes no ambiente de produção (estão em `.env.local`, que é gitignored).
3. (Opcional) Configurar um cron/worker para expirar gerações presas sem depender do polling (§3.4).